Árboles Binarios: Guía Teórico-Práctica

> Guía completa para entender la estructura de datos, su implementación en Python y el análisis de complejidad, partiendo del archivo `arbol_answer.ipynb`.

Tabla de Contenidos

1. Árboles como Estructura de Datos No Lineal
2. Árboles Binarios: Definición y Propiedades
3. Operaciones Fundamentales
4. Recorridos de un Árbol
5. La Clase `Nodo`: El Átomo de Todo Árbol
6. La Clase `AB`: La Base Reutilizable
7. Árboles Binarios de Búsqueda (ABB)
8. Recursividad: Concepto Clave
9. La Clase `ABB`: Implementación Detallada
10. Árboles AVL: Árboles Balanceados
11. La Clase `ArbolAVL`: Implementación Detallada
12. Análisis de Complejidad
13. Jerarquía de Clases: El Diseño Completo

---

1. Árboles como Estructura de Datos No Lineal

Las estructuras de datos se dividen en dos grandes familias: **lineales** y **no lineales**.

En las estructuras **lineales** (arreglos, listas enlazadas, pilas, colas), los elementos se organizan en una secuencia donde cada elemento tiene exactamente un predecesor y un sucesor. Se recorren de inicio a fin de forma unidireccional.

Los **árboles** rompen con ese esquema. Son estructuras **jerárquicas** donde un elemento puede tener múltiples sucesores (llamados *hijos*), pero solo un predecesor (llamado *padre*). Esto los hace ideales para representar:

- Sistemas de archivos (carpetas que contienen subcarpetas)

- Estructuras organizacionales (jerarquías de empleados)
- Expresiones matemáticas (`(a + b) * c`)
- Algoritmos de búsqueda eficiente

Vocabulario esencial

Término	Significado
Raíz	El único nodo sin padre; punto de entrada al árbol
Nodo	Cada elemento del árbol; contiene un dato y referencias a sus hijos
Hoja	Nodo sin hijos
Nodo interno	Nodo que tiene al menos un hijo
Padre	El nodo del que desciende otro
Hijo	Nodo que desciende directamente de otro
Hermanos	Nodos que comparten el mismo padre
Nivel	Distancia de un nodo a la raíz (la raíz está en nivel 0)
Altura	Número de nodos en el camino más largo desde un nodo hasta una hoja
Subárbol	Cualquier nodo del árbol, junto con todos sus descendientes

---

2. Árboles Binarios: Definición y Propiedades

Un **árbol binario** es un árbol donde cada nodo tiene **a lo más dos hijos**: el hijo izquierdo y el hijo derecho. Esta restricción lo hace computacionalmente manejable y muy poderoso.

Propiedades clave

Propiedad 1 — Número máximo de nodos por nivel: Un árbol binario con `k` niveles puede tener como máximo `2^k` nodos en ese nivel (nivel 0 tiene 1, nivel 1 tiene 2, nivel 2 tiene 4, etc.).

Propiedad 2 — Número máximo de nodos total: Un árbol binario de altura `h` tiene a lo más `2^h - 1` nodos.

Propiedad 3 — Altura mínima: Para n nodos, la altura mínima posible es $\lfloor \log_2(n) \rfloor + 1$. Esta es la altura que logra un árbol **perfectamente balanceado**.

Propiedad 4 — Altura máxima (degeneración): Un árbol puede degenerar hasta tener altura n si todos los nodos forman una cadena lineal (cada nodo tiene solo un hijo). En ese caso se comporta exactamente como una lista enlazada.

Tipos de árboles binarios

- **Árbol lleno:** Cada nodo tiene 0 o 2 hijos (nunca 1).
- **Árbol completo:** Todos los niveles están llenos excepto posiblemente el último, que se llena de izquierda a derecha.
- **Árbol perfecto:** Todos los nodos internos tienen exactamente 2 hijos y todas las hojas están al mismo nivel.
- **Árbol degenerado:** Cada nodo tiene a lo más 1 hijo — degenera en lista enlazada.

---

3. Operaciones Fundamentales

Las operaciones que se realizan sobre árboles binarios son:

Inserción: Agregar un nuevo nodo al árbol. En un árbol binario sin orden (clase `AB`) puede insertarse en cualquier posición. En árboles con propiedades específicas (ABB, AVL) la posición está determinada por reglas.

Búsqueda: Localizar un nodo que contiene un dato específico. Recorre el árbol comparando valores.

Eliminación (borrar): Quitar un nodo del árbol manteniendo la estructura y propiedades del mismo.

Eliminación total (elimina): Vaciar el árbol completo de forma inmediata.

Recorrido: Visitar todos los nodos del árbol en un orden determinado (enorden, preorden, postorden).

Altura: Determinar cuántos niveles tiene el árbol desde un nodo dado.

Nivel: Determinar a qué profundidad se encuentra un nodo específico.

---

4. Recorridos de un Árbol

Un recorrido es una forma sistemática de visitar todos los nodos del árbol exactamente una vez. Los tres recorridos clásicos difieren solo en **cuándo se procesa el nodo raíz** respecto a sus subárboles.

Preorden (Raíz → Izquierda → Derecha)

Primero se visita el nodo actual, luego el subárbol izquierdo, luego el derecho. Es útil para copiar o serializar la estructura del árbol.

```
preorden(nodo):
    si nodo no es None:
        procesar(nodo)          ← primero yo
        preorden(nodo.izq)
        preorden(nodo.der)
```

Enorden (Izquierda → Raíz → Derecha)

Primero el subárbol izquierdo, luego el nodo actual, luego el derecho. En un ABB, este recorrido produce los datos **en orden ascendente** — es su propiedad más importante.

```
enorden(nodo):
    si nodo no es None:
        enorden(nodo.izq)
        procesar(nodo)          ← en medio
        enorden(nodo.der)
```

Postorden (Izquierda → Derecha → Raíz)

Primero los hijos, luego el nodo actual. Es útil para eliminar el árbol o evaluar expresiones matemáticas.

```
postorden(nodo):
    si nodo no es None:
        postorden(nodo.izq)
        postorden(nodo.der)
        procesar(nodo)          ← último
```

La clase `AB` implementa los tres recorridos como métodos reutilizables por todas las subclases.

5. La Clase `Nodo`: El Átomo de Todo Árbol

¿Por qué se empieza por `Nodo`?

Todo árbol está construido de nodos. Antes de poder hablar de un árbol, se necesita definir de qué están hechas sus piezas. La clase `Nodo` es la **unidad mínima indivisible** de cualquier árbol binario.

```python
class Nodo(object):
    def __init__(self, dato):
        self.izq = None      # referencia al hijo izquierdo
        self.dato = dato     # el valor almacenado
        self.der = None      # referencia al hijo derecho
        self.padre = None    # referencia al nodo padre
```

Anatomía del nodo

Cada nodo tiene exactamente cuatro atributos:

- `dato`: El valor que almacena el nodo. Puede ser un número, una cadena, cualquier objeto Python.
- `izq`: Una referencia al nodo hijo izquierdo. Si no tiene hijo izquierdo, vale `None`.
- `der`: Una referencia al nodo hijo derecho. Si no tiene hijo derecho, vale `None`.
- `padre`: Una referencia al nodo que lo contiene. La raíz tiene `padre = None`.

Visualmente, un nodo se representa así:

```
        [padre]
           |
    [izq] [dato] [der]
```

¿Por qué `Nodo` es la base de ABB y AVL?

`Nodo` no es una *superclase* de `ABB` o `AVL` en sentido de herencia de clases, sino que es el **componente estructural** del que están hechos. La relación es de **composición**: los árboles *contienen* nodos, no *son* nodos.

Sin embargo, `Nodo` es la misma clase que usan `AB`, `ABB` y `ArbolAVL`. Esto es posible porque el nodo en sí no tiene lógica de ordenamiento; esa lógica vive en las clases del árbol. El nodo solo sabe que tiene un padre, un hijo izquierdo y un hijo derecho.

Métodos auxiliares del nodo

```python
def tieneHijos(self):
    return self.izq is not None and self.der is not None
```

Retorna `True` si el nodo tiene **ambos** hijos. Útil para decidir casos en eliminación.

```python
def esIzq(self):
    return self.padre.izq == self
```

Permite saber si el nodo es hijo izquierdo de su padre, sin tener que acceder al padre externamente.

```python
def esDer(self):
    return self.padre.der == self
```

Idem para hijo derecho.

---

6. La Clase `AB`: La Base Reutilizable

¿Qué es y para qué sirve?

La clase `AB` (Árbol Binario) implementa la **funcionalidad genérica** que comparten todos los tipos de árboles binarios, sin importar si tienen o no propiedades de ordenamiento o balanceo.

```python
class AB(object):
    def __init__(self, dato):
        self.raiz = Nodo(dato)
```

Al crearse un árbol, inmediatamente se construye un `Nodo` que será la raíz. El árbol no almacena datos directamente: almacena una **referencia a la raíz**, y desde ahí se puede navegar hacia cualquier parte del árbol.

Por qué los métodos de `AB` son reutilizables

`AB` define operaciones que son **independientes del orden o balance** del árbol. Son operaciones sobre la *estructura* en sí misma:

`elimina()` — Elimina todo el árbol en O(1):

```python
def elimina(self):
    self.raiz = None
```

Basta con cortar la referencia a la raíz. Python libera automáticamente la memoria de los nodos a través de su recolector de basura. Esta simplicidad es posible porque el árbol existe solo mientras algo apunte a su raíz.

`nivel(nodo)` — Calcula a qué profundidad está un nodo:

```python
def nivel(self, nodo):
    if nodo is None:
        return -1
    else:
        return 1 + self.nivel(nodo.padre)
```

Sube hacia la raíz siguiendo los punteros `padre` y cuenta los saltos. La raíz tiene nivel 0 (su padre es `None`, que retorna -1, y 1 + (-1) = 0). Este método no depende de si el árbol está ordenado o balanceado.

`altura(nodo)` — Calcula la altura del subárbol desde un nodo:

```python
def altura(self, nodo):
    if nodo is None:
        return 0
    else:
        return 1 + max(self.altura(nodo.izq), self.altura(nodo.der))
```

Baja recursivamente hacia las hojas y toma el camino más largo. Una hoja tiene altura 1 ($1 + \max(0, 0)$). Un árbol vacío tiene altura 0. Este método es **crítico para el AVL**, ya que calcula el factor de balance.

`recorrido_enorden`, `recorrido_preorden`, `recorrido_postorden` — Los tres recorridos clásicos. Al estar en `AB`, cualquier subclase los hereda sin necesidad de redefinirlos.

`insertar_nodo` — La inserción aleatoria en un árbol binario sin orden. La clase `AB` inserta un nuevo dato de forma aleatoria (cara o cruz decidido con `random.getrandbits(1)`), ya que en un árbol binario genérico no hay una regla de posicionamiento.

Herencia: cómo `AB` sirve de base

```
AB  (árbol binario genérico)
 └── ABB  (añade la propiedad de orden)
      └── ArbolAVL  (añade la propiedad de balance)
```

Cada subclase **hereda** todos los métodos de su clase padre sin tener que reimplementarlos. Cuando `ABB` o `ArbolAVL` llaman a `self.altura(nodo)`, están usando el método definido en `AB`. Solo redefinen (`override`) los métodos que cambian de comportamiento, principalmente `insertar` y `borrar`.

---

7. Árboles Binarios de Búsqueda (ABB)

Definición y propiedad fundamental

Un **Árbol Binario de Búsqueda** (ABB o BST, *Binary Search Tree*) es un árbol binario que cumple una propiedad adicional de **orden**:

> Para cada nodo `N` del árbol:
> - Todos los nodos en el **subárbol izquierdo** de `N` tienen un valor **menor** que `N.dato`
> - Todos los nodos en el **subárbol derecho** de `N` tienen un valor **mayor** que `N.dato`
> - No se permiten **datos duplicados**

Esta propiedad se aplica **recursivamente** a cada subárbol.

Por qué esta propiedad es poderosa

Considera buscar el valor `14` en este árbol:

```
        15
       /  \
     10    20
    /  \   / \
   8   14 -  35
  / \          \
 0   -          99
```

Sin la propiedad de orden, habría que examinar todos los nodos (búsqueda lineal O(n)). Con la propiedad ABB:

1. ¿Es 14 igual a 15? No. ¿Es 14 < 15? Sí → ir a la izquierda.
2. ¿Es 14 igual a 10? No. ¿Es 14 < 10? No → ir a la derecha.
3. ¿Es 14 igual a 14? ¡Sí! Encontrado.

Solo se examinaron 3 nodos en lugar de recorrer todo el árbol. En un árbol balanceado con `n` nodos, esto toma O(log n) pasos.

El recorrido enorden de un ABB produce datos ordenados

Esta es una consecuencia directa de la propiedad de orden. Si se hace enorden (Izquierda → Raíz → Derecha) sobre el árbol anterior: 0 → 8 → 10 → 14 → **15** → 20 → 35 → 99

Los datos aparecen en orden ascendente. Esto es una forma de verificar que un árbol cumple la propiedad ABB.

---

8. Recursividad: Concepto Clave

Antes de implementar `ABB`, es necesario entender la **recursividad**, ya que casi todas las operaciones sobre árboles son recursivas por naturaleza.

¿Qué es la recursividad?

Una función es **recursiva** cuando se llama a sí misma para resolver una versión más pequeña del mismo problema. Toda función recursiva debe tener:

1. **Caso base:** Una condición que detiene la recursión y devuelve un resultado directo.
2. **Caso recursivo:** Una llamada a sí misma con un subproblema más pequeño.

¿Por qué los árboles y la recursividad van de la mano?

La definición de árbol es recursiva: un árbol es un nodo raíz cuyos hijos son a su vez árboles. Esta estructura se refleja naturalmente en algoritmos recursivos:

```
# Ejemplo conceptual: calcular suma de todos los valores
suma(nodo):
    si nodo es None:
        return 0                                  ← caso base
    return nodo.dato + suma(nodo.izq) + suma(nodo.der)   ← caso recursivo
```

Cada llamada trabaja en un subárbol más pequeño hasta llegar a `None` (árbol vacío).

La pila de llamadas (call stack)

Cuando Python ejecuta una función recursiva, guarda el estado de cada llamada en la **pila de llamadas**:

```
insertar(raiz, 14)
  └── insertar(nodo_10, 14)          ← 14 > 10, ir a la derecha
        └── insertar(None, 14)       ← caso base: crear nodo
              return Nodo(14)
        nodo_10.der = Nodo(14)
        return nodo_10
  raiz.izq = nodo_10 (actualizado)
  return raiz
```

La profundidad de esta pila es igual a la **altura del árbol**. En un árbol balanceado es O(log n); en uno degenerado puede ser O(n), lo que podría causar un desbordamiento de pila (`RecursionError`) en árboles muy grandes.

---

9. La Clase `ABB`: Implementación Detallada

```python
class ABB(AB):
```

`ABB` hereda de `AB`. Esto significa que `ABB` tiene acceso a todos los métodos de `AB` (`elimina`, `altura`, `nivel`, los recorridos) sin reescribirlos. Solo define nuevos métodos o sobreescribe `insertar`.

Método `busqueda`

```python
def busqueda(self, nodo, dato):
    if nodo is None:
        return None
    if dato == nodo.dato:
        return nodo
    elif dato < nodo.dato:
        return self.busqueda(nodo.izq, dato)
    else:
        return self.busqueda(nodo.der, dato)
```

La búsqueda explota directamente la propiedad de orden del ABB:

- **Caso base 1:** `nodo is None` — hemos llegado a un lugar vacío, el dato no existe.
- **Caso base 2:** `dato == nodo.dato` — encontramos el dato, retornamos el nodo.
- **Caso recursivo izquierdo:** Si el dato buscado es menor que el actual, el dato solo puede estar en el subárbol izquierdo.
- **Caso recursivo derecho:** Si el dato buscado es mayor, solo puede estar en el subárbol derecho.

En cada nivel se descarta **la mitad del árbol** (en un árbol balanceado). Es exactamente la misma lógica que la búsqueda binaria en un arreglo ordenado.

Por qué `insertar` necesita la función auxiliar `_insertar_abb`

```python
```

```python
def insertar(self, dato):
    self.raiz = self._insertar_abb(self.raiz, dato, None)

def _insertar_abb(self, nodo, dato, padre):
    if nodo is None:
        nuevo = Nodo(dato)
        nuevo.padre = padre
        return nuevo
    if dato == nodo.dato:
        return nodo
    elif dato < nodo.dato:
        nodo.izq = self._insertar_abb(nodo.izq, dato, nodo)
    else:
        nodo.der = self._insertar_abb(nodo.der, dato, nodo)
    return nodo
```

La función pública `insertar` solo recibe un `dato`. Pero la inserción recursiva necesita saber **en qué nodo se está trabajando actualmente** y **quién es el padre del nodo nuevo**. Pasar el nodo actual y el padre como parámetros es la técnica estándar para escribir recursión sobre árboles.

`_insertar_abb` aplica el patrón clásico de recursión sobre árboles:

1. Si el nodo actual es `None`, es el lugar correcto para insertar — se crea el nuevo nodo con su padre ya asignado.
2. Si el dato ya existe, se retorna el nodo sin cambios (no se permiten duplicados).
3. Si el dato es menor, se inserta en el subárbol izquierdo; si es mayor, en el derecho.
4. Al retornar, se reconecta el subárbol modificado: `nodo.izq = ...` o `nodo.der = ...`.

El truco clave está en `self.raiz = self._insertar_abb(self.raiz, dato, None)`. Al retornar el nodo en cada nivel de la recursión, se garantiza que las conexiones del árbol queden correctamente actualizadas.

Por qué es necesaria la función `_minimo`

```python
```

```python
def _minimo(self, nodo):
    actual = nodo
    while actual.izq is not None:
        actual = actual.izq
    return actual
```

En un ABB, el nodo con el valor mínimo de cualquier subárbol siempre es el nodo **más a la izquierda** de ese subárbol. Esta función lo encuentra siguiendo los hijos izquierdos hasta llegar a una hoja o a un nodo sin hijo izquierdo.

`_minimo` es necesaria para implementar el **caso de tres: eliminación de un nodo con dos hijos**. Cuando se borra un nodo que tiene tanto hijo izquierdo como hijo derecho, no se puede simplemente quitarlo; hay que poner algo en su lugar. La elección correcta es el **sucesor inorden**: el menor valor que sea mayor que el nodo eliminado. Ese sucesor siempre está en la posición más a la izquierda del subárbol derecho del nodo a eliminar.

La recursividad en `_borrar_abb`

```python
```

```python
def _borrar_abb(self, nodo, dato):
    if nodo is None:
        return None

    if dato < nodo.dato:
        nodo.izq = self._borrar_abb(nodo.izq, dato)
        if nodo.izq is not None:
            nodo.izq.padre = nodo
    elif dato > nodo.dato:
        nodo.der = self._borrar_abb(nodo.der, dato)
        if nodo.der is not None:
            nodo.der.padre = nodo
    else:
        # Nodo encontrado: tres casos
        if nodo.izq is None and nodo.der is None:
            return None                         # Caso 1: hoja
        elif nodo.izq is None:
            nodo.der.padre = nodo.padre
            return nodo.der                     # Caso 2a: solo hijo derecho
        elif nodo.der is None:
            nodo.izq.padre = nodo.padre
            return nodo.izq                     # Caso 2b: solo hijo izquierdo
        else:
            sucesor = self._minimo(nodo.der)    # Caso 3: dos hijos
            nodo.dato = sucesor.dato
            nodo.der = self._borrar_abb(nodo.der, sucesor.dato)
            if nodo.der is not None:
                nodo.der.padre = nodo
    return nodo
```

La eliminación tiene tres casos que la recursión maneja de forma elegante:

Caso 1 — Nodo hoja: No tiene hijos. Simplemente se elimina retornando `None`. El nodo padre, al recibir `None` como resultado de la llamada recursiva, actualizará su referencia hijo a `None`.

Caso 2 — Nodo con un solo hijo: Se reemplaza el nodo por su único hijo. Se devuelve el hijo y el padre del nodo eliminado se conecta directamente a ese hijo.

Caso 3 — Nodo con dos hijos: Este es el caso complejo. No se puede eliminar el nodo directamente porque rompería el árbol en dos partes. La solución:

1. Se encuentra el **sucesor inorden** (el mínimo del subárbol derecho) con `_minimo`.
2. Se **copia el dato del sucesor** al nodo que queremos eliminar.
3. Se **elimina el sucesor** del subárbol derecho (el sucesor siempre tiene a lo más un hijo derecho, por lo que su eliminación es trivial — caso 1 o 2).

La recursión en borrar es ascendente: baja hasta encontrar el nodo a eliminar y, al volver hacia arriba, reconecta el árbol correctamente. La línea `return nodo` al final de cada rama garantiza que los nodos superiores reciban la nueva versión de su subárbol modificado.

---

10. Árboles AVL: Árboles Balanceados

¿Qué es un árbol AVL?

Un **árbol AVL** (Adelson-Velsky y Landis, 1962) es un **Árbol Binario de Búsqueda con la propiedad adicional de auto-balanceo**.

La propiedad que define a un AVL es:

> Para cada nodo del árbol, la diferencia de alturas entre su subárbol izquierdo y su subárbol derecho es de **a lo más 1** en valor absoluto.

Esta diferencia se llama **factor de balance** (FB):

```
Factor de Balance(nodo) = altura(subárbol izquierdo) - altura(subárbol
derecho)

Condición AVL: |FB(nodo)| ≤ 1  para todo nodo del árbol
```

Si el FB es 0, el nodo está perfectamente balanceado. Si es 1 o -1, es aceptable. Si es 2 o -2, el árbol **viola la propiedad AVL** y debe rebalancearse mediante rotaciones.

¿Por qué un AVL es un subconjunto del ABB?

Un árbol AVL **es** un ABB: cumple la propiedad de orden (izquierda menor, derecha mayor, sin duplicados). La diferencia es que **además** cumple la propiedad de balance.

Por eso `ArbolAVL` **hereda de** `ABB` y no de `AB` directamente. Al heredar de `ABB`, obtiene:

- La clase `Nodo` como estructura de nodo.
- Los métodos de `AB` (`altura`, `nivel`, `elimina`, los recorridos).
- Los métodos de `ABB` (`busqueda`, `_insertar_abb`, `_minimo`, `_borrar_abb`).

El AVL **reutiliza** `_insertar_abb` y `_borrar_abb` como pasos intermedios dentro de sus propias versiones (`_insertar_avl` y `_borrar_avl`), añadiendo el rebalanceo al final de cada llamada recursiva.

No es que el AVL *sea un subconjunto* del ABB en sentido estricto, sino que **todo AVL es un ABB válido** (relación de inclusión): el conjunto de todos los AVL está contenido en el conjunto de todos los ABB. Un ABB no necesariamente es un AVL, pero un AVL siempre es un ABB.

¿Por qué importa el balanceo?

En un ABB degenerado (insertando valores en orden: 1, 2, 3, 4, 5...), el árbol se convierte en una lista enlazada y la búsqueda tarda O(n). Con un AVL, esto **nunca sucede**: las rotaciones garantizan que la altura siempre sea O(log n), y por tanto la búsqueda siempre tarda O(log n).

```
ABB degenerado          vs.          AVL balanceado
    1                                      3
     \                                   /   \
      2                               1       4
       \                               \     / \
        3                               2 -   5
         \
          4
           \
            5
```

---

11. La Clase `ArbolAVL`: Implementación Detallada

```python
class ArbolAVL(ABB):
```

La función `_factor_balance`

```python
```

```python
def _factor_balance(self, nodo):
    if nodo is None:
        return 0
    return self._altura_nodo(nodo.izq) - self._altura_nodo(nodo.der)
```

Calcula la diferencia de alturas entre el subárbol izquierdo y el derecho:

- FB > 0: el subárbol izquierdo es más alto ("inclinado a la izquierda").
- FB < 0: el subárbol derecho es más alto ("inclinado a la derecha").
- FB = 0: perfectamente balanceado.
- |FB| > 1: violación AVL, hay que rotar.

Las rotaciones: `_rotar_derecha` y `_rotar_izquierda`

Una **rotación** es una operación que reestructura localmente el árbol para reducir su altura, manteniendo la propiedad de orden del ABB.

Rotación simple a la derecha (`_rotar_derecha`)

Se usa cuando el subárbol **izquierdo** está demasiado alto (FB = +2) y la inserción fue en el subárbol izquierdo del hijo izquierdo (**caso LL**).

```
    Antes de rotar:          Después de rotar:

         z (FB=+2)                  y
        / \                        / \
       y   T4          →          x   z
      / \                            / \
     x   T3                       T3  T4
```

`y` sube y `z` baja a la derecha. `T3` (el subárbol derecho de `y`) pasa a ser el subárbol izquierdo de `z`. La propiedad de orden se mantiene porque:

- Los valores de `T3` son mayores que `y` y menores que `z`, así que pueden ser el hijo izquierdo de `z`.

```python
```

```python
def _rotar_derecha(self, z):
    y = z.izq
    T3 = y.der
    y.der = z        # z pasa a ser hijo derecho de y
    z.izq = T3       # T3 pasa a ser hijo izquierdo de z
    # Actualizar punteros padre
    y.padre = z.padre
    z.padre = y
    if T3 is not None:
        T3.padre = z
    return y         # y es la nueva raíz del subárbol
```

Rotación simple a la izquierda (`_rotar_izquierda`)

Espejo de la anterior. Se usa cuando el subárbol **derecho** está demasiado alto (FB = -2) y la inserción fue en el subárbol derecho del hijo derecho (**caso RR**).

```
    Antes de rotar:          Después de rotar:


       z (FB=-2)                    y
      / \                          / \
     T1   y            →          z   x
         / \                      / \
        T2   x                   T1  T2
```

La función `_rebalancear`: Los cuatro casos AVL

```python
```

```python
def _rebalancear(self, nodo):
    fb = self._factor_balance(nodo)

    if fb > 1 and self._factor_balance(nodo.izq) >= 0:
        return self._rotar_derecha(nodo)          # Caso LL

    if fb > 1 and self._factor_balance(nodo.izq) < 0:
        nodo.izq = self._rotar_izquierda(nodo.izq)
        return self._rotar_derecha(nodo)          # Caso LR

    if fb < -1 and self._factor_balance(nodo.der) <= 0:
        return self._rotar_izquierda(nodo)        # Caso RR

    if fb < -1 and self._factor_balance(nodo.der) > 0:
        nodo.der = self._rotar_derecha(nodo.der)
        return self._rotar_izquierda(nodo)        # Caso RL

    return nodo   # ya está balanceado
```

Los cuatro casos surgen porque el desbalanceo puede ocurrir en cuatro configuraciones distintas dependiendo de **hacia dónde** se inclinó el árbol y **de qué lado del hijo** viene el exceso:

Caso LL (Left-Left): El árbol se inclinó a la izquierda (FB = +2) y el hijo izquierdo también está inclinado a la izquierda (FB hijo izq ⩾ 0). Solución: una sola rotación a la derecha sobre el nodo desbalanceado.

Caso LR (Left-Right): El árbol se inclinó a la izquierda (FB = +2) pero el hijo izquierdo está inclinado a la derecha (FB hijo izq < 0). Si se aplicara directamente una rotación derecha, el resultado seguiría desbalanceado. Solución: primero rotar el hijo izquierdo a la izquierda (convirtiendo LR en LL), y luego rotar el nodo desbalanceado a la derecha. Son dos rotaciones.

Caso RR (Right-Right): Espejo del caso LL. El árbol se inclinó a la derecha (FB = -2) y el hijo derecho también está a la derecha (FB hijo der ⩽ 0). Solución: una rotación a la izquierda.

Caso RL (Right-Left): Espejo del caso LR. Árbol inclinado a la derecha (FB = -2) pero el hijo derecho inclinado a la izquierda (FB hijo der > 0). Solución: primero rotar el hijo derecho a la derecha (convirtiendo RL en RR), y luego rotar el nodo desbalanceado a la izquierda. Dos rotaciones.

La necesidad de los cuatro casos radica en que una rotación simple solo funciona cuando el "peso extra" está **en el mismo lado** que la inclinación. Cuando está en el lado contrario

(casos LR y RL), se necesita una rotación preparatoria que alinee el peso.

Inserción en el AVL: cómo el rebalanceo se propaga hacia arriba

```python
def _insertar_avl(self, nodo, dato, padre):
    if nodo is None:
        nuevo = Nodo(dato)
        nuevo.padre = padre
        return nuevo
    if dato < nodo.dato:
        nodo.izq = self._insertar_avl(nodo.izq, dato, nodo)
        if nodo.izq is not None:
            nodo.izq.padre = nodo
    elif dato > nodo.dato:
        nodo.der = self._insertar_avl(nodo.der, dato, nodo)
        if nodo.der is not None:
            nodo.der.padre = nodo
    else:
        return nodo  # duplicado
    nodo = self._rebalancear(nodo)   # ← rebalancear al regresar
    return nodo
```

La clave está en que `_rebalancear` se llama **en cada nivel al regresar de la recursión**. Cuando se inserta un nodo nuevo, los cambios de altura se propagan hacia arriba por el árbol. En cada nodo del camino de regreso se verifica si el factor de balance se violó y se corrige de inmediato.

Esto garantiza que tras cualquier inserción, el árbol quede en un estado AVL válido.

Eliminación en el AVL: idéntica lógica, mismo rebalanceo

`_borrar_avl` es prácticamente idéntico a `_borrar_abb` con una sola diferencia: al final de cada llamada recursiva, antes de retornar, se llama a `_rebalancear(nodo)`. La eliminación puede desbalancear el árbol tanto como la inserción, por lo que el mismo mecanismo de propagación ascendente aplica.

---

12. Análisis de Complejidad

La **complejidad temporal** mide cuánto tiempo tarda un algoritmo en función del tamaño de la entrada `n`. La **complejidad espacial** mide cuánta memoria adicional usa.

La notación **O(...)** describe el **peor caso**; en algoritmos sobre árboles, `n` es el número de nodos.

Tabla comparativa ABB vs. AVL

Operación	ABB promedio	ABB peor caso	AVL (siempre)
`busqueda`	O(log n)	O(n)	O(log n)
`insertar`	O(log n)	O(n)	O(log n)
`borrar`	O(log n)	O(n)	O(log n)
`elimina`	O(1)	O(1)	O(1)
Espacio (pila)	O(log n)	O(n)	O(log n)

¿Por qué O(log n) y no O(1)?

Las operaciones de búsqueda, inserción y borrado en un árbol balanceado requieren descender desde la raíz hasta una hoja (o casi). La altura de un árbol balanceado con `n` nodos es $\lfloor \log_2(n) \rfloor$. Por eso el número de pasos es proporcional a `log n`.

Intuitivamente: cada vez que bajamos un nivel en el árbol, descartamos aproximadamente la mitad de los nodos restantes. Esto es igual a la búsqueda binaria en un arreglo ordenado.

¿Por qué O(n) en el peor caso del ABB?

Si se insertan los valores `1, 2, 3, 4, 5, ..., n` en ese orden en un ABB, cada valor va siempre al subárbol derecho del anterior, formando una cadena:

```
1
 \
  2
   \
    3
     \
      4
```

La altura es `n`, no `log n`. Buscar el último elemento requiere recorrer los `n` nodos uno a uno. El ABB sin balanceo no garantiza que esto no ocurra.

¿Por qué el AVL garantiza O(log n)?

Las rotaciones del AVL mantienen `|FB| ≤ 1` en cada nodo. Puede demostrarse matemáticamente que un árbol AVL con `n` nodos tiene altura a lo más `1.44 * log₂(n)`. Esto es O(log n), garantizado sin importar el orden de inserción.

Complejidad de `elimina`: O(1)

```python
def elimina(self):
    self.raiz = None
```

Una sola asignación, independiente de `n`. O(1) en tiempo y espacio. La memoria de los nodos la libera el recolector de basura de Python, pero eso no cuenta como parte de la complejidad del algoritmo.

Complejidad espacial de la recursión

Las funciones `_insertar_abb`, `_borrar_abb`, `_insertar_avl`, `_borrar_avl`, `busqueda` y `altura` son todas recursivas. Cada llamada recursiva agrega un **frame** a la pila de llamadas de Python, consumiendo memoria proporcional a la profundidad de la recursión.

- En un árbol balanceado (AVL o ABB bien balanceado): profundidad O(log n) → espacio O(log n).
- En un ABB degenerado: profundidad O(n) → espacio O(n).

---

13. Jerarquía de Clases: El Diseño Completo

Diagrama de herencia

```
object
  └── Nodo          (estructura de datos del nodo)
  └── AB            (árbol binario genérico)
        └── ABB     (árbol binario de búsqueda)
              └── ArbolAVL  (árbol balanceado auto-rebalanceable)
```

Qué hereda cada clase

`ABB` hereda de `AB`:

- `__init__` (constructor con raíz)
- `elimina` (vaciar árbol en O(1))
- `nivel` (profundidad de un nodo)
- `altura` (altura de un subárbol)
- `insertar_nodo` (inserción aleatoria — aunque `ABB` sobreescribe `insertar`)
- `recorrido_enorden`, `recorrido_preorden`, `recorrido_postorden`

`ArbolAVL` hereda de `ABB` (y transitivamente de `AB`):

- Todo lo anterior
- `busqueda` (la búsqueda no cambia entre ABB y AVL, ya que la propiedad de orden es la misma)
- `_insertar_abb` (se reutiliza como paso de inserción estándar dentro de `_insertar_avl`)
- `_minimo` (se reutiliza en `_borrar_avl` para el caso de dos hijos)
- `_borrar_abb` (se reutiliza conceptualmente; `ArbolAVL` define `_borrar_avl` que replica su lógica y añade rebalanceo)

La consistencia entre teoría y código

El código es consistente con la teoría en cada decisión de diseño:

La **herencia** refleja que AVL ⊂ ABB ⊂ AB en términos matemáticos: todo AVL es un ABB, y todo ABB es un árbol binario. La jerarquía de clases replica exactamente esta relación de conjuntos.

La **recursividad** es la consecuencia directa de que los árboles tienen definición recursiva. No es una elección arbitraria; es la forma más natural y correcta de operar sobre ellos.

La **función auxiliar privada** (`_insertar_abb`, `_borrar_abb`, `_insertar_avl`, `_borrar_avl`) es necesaria porque la API pública (`insertar(dato)`, `borrar(dato)`) no puede exponer los parámetros internos de la recursión (el nodo actual y el padre). El prefijo `_` en Python indica por convención que son métodos de uso interno.

El **rebalanceo al regresar** en las funciones AVL es consistente con la teoría: el desbalanceo se detecta y corrige en el camino de vuelta de la recursión, propagando el balanceo desde la hoja hasta la raíz. Esto garantiza que el árbol entero quede AVL-válido tras cada operación.

La **función** `_minimo` es consistente con la propiedad de orden del ABB: el menor valor de cualquier subárbol siempre es el nodo más profundo a la izquierda. Esta propiedad

estructural se explota directamente en la eliminación.

Los **cuatro casos de rotación** son la enumeración exhaustiva de todas las configuraciones posibles de desbalanceo: izquierda-izquierda, izquierda-derecha, derecha-derecha, derecha-izquierda. No hay más casos posibles, y el código los cubre todos.

Esta guía cubre la totalidad del contenido del archivo `arbol_answer.ipynb`*, articulando la teoría detrás de cada decisión de diseño e implementación.*